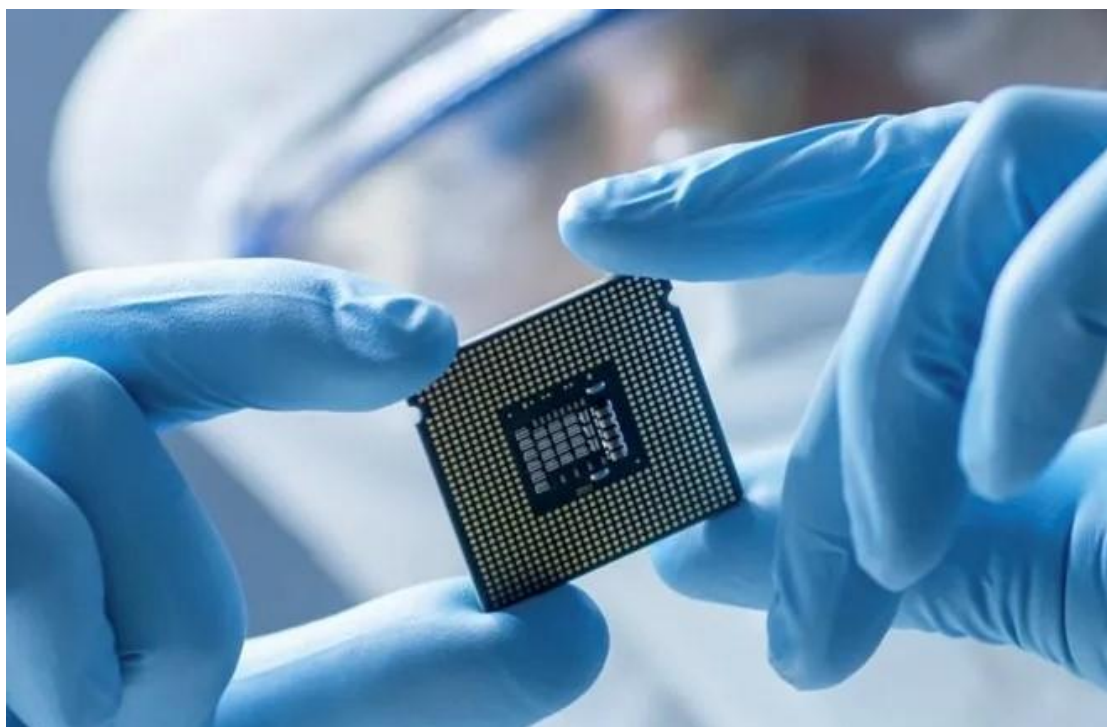




軟硬體協同設計

HW2

林承羿 | 01257027

基礎題與加分題

*在 LCD 中心點畫圓(實心空心皆可)，初始狀態圓不動，隨功能鍵變化

*各按鍵功能：

KEY0 – 上下彈跳(觸邊界反彈)

KEY0 – 左右彈跳(觸邊界反彈)

KEY0 – 圓變大(大多少自由發揮)

KEY0 – 圓變小(小多少自由發揮)

} ← 加分題

*上下左右移動時不需先回到初始位置

程式碼截圖與說明

```
10 // radius of circle
11 u8 r = 12;
12 u16 x = 0, y = 0;
13 u8 rightFg = 0, leftFg = 0, upFg = 0, downFg = 0, smallFg = 0, bigFg = 0;
```

以下聲明，本人在此課程使用英文註解只是想避開中文編碼問題，並非交由科技的力量完成作業的寫作，作業上的東西只要不是模板都是自己敲出來的，如看到中文註解，只是代表我從模板搬運過來懶得改。

以下說明基礎題、加分題共同使用到的變數

r	圓形半徑
x,y	圓心座標
XXFg	向右、左、上、下，縮小、放大的 flag，指示誰可以 enable

```

int main(void) {
    KEY_Init();
    EXTI_Init();
    delay_init(168);
    // why this fucking shit can enable LCD?
    uart_init(115200);
    // counter is 84M, 84M/8400=10kHz, 10kHz*500=50ms
    TIM3_Int_Init(500-1,8400-1);
    // counter is 84M, 84M/8400=10kHz, 10kHz*5000=500ms (addition)
    TIM4_Int_Init(5000-1,8400-1);
    // counter is 84M, 84M/8400=10kHz, 10kHz*100=10ms (addition)
    TIM5_Int_Init(100-1,8400-1);
    LCD_Init();
    LED_Init();
    // set background equal "black"
    LCD_Clear(BLACK);
    // set pencil color equal "red"
    POINT_COLOR=RED;
    // set default x, y at middle of LCD
    x = (lcddev.width)/2, y = (lcddev.height)/2;
}

```

上圖先是做了一堆的初始化，主要需要在乎的有 key、uart、timer 3、LCD，使 button 按下可被捕捉到信號，uart、LCD 必須綁在一起，我也不知道為啥，但 uart 沒寫 LCD 就是不會亮，我看了腳位很久，看了 code 也挺久，但不知道為啥，猜測是 uart 設定到了 LCD 需要初始化的東西。Timer 3 很重要的原因是我把所有判斷邏輯與定時更新螢幕的工作全部塞在 timer 3 中斷裡面。螢幕的初始化為黑背景 (我還不想瞎掉的部分)，紅色圓圈。位置為螢幕正中心。

```

138 void EXTI0_IRQHandler(void) {
139     // debounce
140     delay_ms(10);
141     if(WK_UP==1) {
142         // remind user the function is working right now
143         if (smallFg == 1) ledFg = 1;
144         else {
145             // turn circle small
146             smallFg = 1, bigFg = 0;
147             // disable any direction
148             leftFg = 0, rightFg = 0, upFg = 0, downFg = 0;
149         }
150     }
151     EXTI_ClearITPendingBit(EXTI_Line0);
152 }

```

```

void EXTI2_IRQHandler(void) {
    delay_ms(10);
    if(KEY2==0) {
        // remind user the function is working right now
        if ((leftFg == 1) || (rightFg == 1)) ledFg = 1;
        else {
            // disable big and small
            smallFg = 0, bigFg = 0;
            // disable up and down
            upFg = 0, downFg = 0;
            // default go right, else go left
            rightFg = 1, leftFg = 0;
        }
    }
    EXTI_ClearITPendingBit(EXTI_Line2);
}

```

```

void EXTI3_IRQHandler(void) {
    delay_ms(10);
    if(KEY1==0) {
        // remind user the function is working right now
        if (bigFg == 1) ledFg = 1;
        else {
            // turn circle big
            smallFg = 0, bigFg = 1;
            // disable any direction
            leftFg = 0, rightFg = 0, upFg = 0, downFg = 0;
        }
    }
    EXTI_ClearITPendingBit(EXTI_Line3);
}

```

```

void EXTI4_IRQHandler(void) {
    delay_ms(10);
    if(KEY0==0) {
        // remind user the function is working right now
        if ((upFg == 1) || (downFg == 1)) ledFg = 1;
        else {
            // disable big and small
            smallFg = 0, bigFg = 0;
            // disable left and right
            leftFg = 0, rightFg = 0;
            // default go up, else go down
            upFg = 1, downFg = 0;
        }
    }
    EXTI_ClearITPendingBit(EXTI_Line4);
}

```

以上四張圖代表的 4 個 button 按下後該有的運算 (來自函示庫實驗 5 外部中斷搬運過來的)。使用此中斷相較 timer 中斷後 polling 每個 button 是否有被按下省下了許多運算資源，是較好的選擇 (OS 中 interrupt vs polling 的問題)。此部分設計的想法是「在乎方向即可，往該方向走下去是 timer interrupt 的工作」。依照老師的要求，移動只要單一方向即可，不會有右上、左下的問題，設定每次 enable 的 flag 只會有一個，因為無法得知上一刻是哪個方向，每次需要 disable 所有 flag 重新 enable 單一的 flag (極端例子是貼著右邊界向下，撞到下邊界時按下了 Key0，試問此狀況是因為向右撞到右邊界抑或是向下撞到下邊界?)。放大縮小也是單一的動作，不會發生邊移動邊放大的情況，因此呈現上才會需要 disable 其他 flag。

移動的方向是有預設的，當 reset 時，只會有一個固定的圓出現在正中心，沒有現在是向左所以下一刻該向右的事情，因此預設如果方向是上下先向上，左右先向右。在此時只能判斷方向，不可以移動圓心，否則可能會發生在先移動後超出邊界，timer 判斷晚一步的情況 (假設已經在最右邊界，又向右，而 timer 中斷未發生，會是晚一步才把圓抓回來)。因此需要嚴格遵守「只管方向」的原則。

對於按鍵的優化有解彈跳 (本人習慣上會做) 及觸發時行為的優化 -- 不會再作動的情況下再重新觸發一次 (意思是假定現在向下，在按下 button 決定上下方向時被預設方向「上」影響，在沒有撞到邊界的情況下發生立刻向上的問題)。


```

61 // timer 3 interrupt
62 void TIM3_IRQHandler(void) {
63     if(TIM_GetITStatus(TIM3,TIM_IT_Update)==SET) {
64         // reset LCD
65         LCD_Clear(BLACK);
66         // enable go right but encounter right border
67         if ((rightFg == 1) && ((x+r)<lcddev.width)) x++;
68         else if (rightFg == 1) {
69             // change direction to left
70             rightFg = 0, leftFg = 1;
71             x--;
72         }
73         // enable go left but encounter left border
74         if ((leftFg == 1) && ((x-r)>0)) x--;
75         else if (leftFg == 1) {
76             // change direction to right
77             rightFg = 1, leftFg = 0;
78             x++;
79         }
80         // enable go up but encounter up border
81         if ((upFg == 1) && ((y+r)<lcddev.height)) y++;
82         else if (upFg == 1) {
83             // change direction to down
84             upFg = 0, downFg = 1;
85             y--;
86         }
87         // enable go down but encounter down border
88         if ((downFg == 1) && ((y-r)>0)) y--;
89         else if (downFg == 1) {
90             // change direction to up
91             upFg = 1, downFg = 0;
92             y++;
93         }
94         // radius turn small (r/2 can't little than font size)
95         if ((smallFg == 1) && ((r*2)>fontSize)) r--;
96         // radius can't bigger than width of LCD
97         if ((bigFg == 1) && ((x+r)<lcddev.width) && ((x-r)>0) && ((y+r)<lcddev.height) && ((y-r)>0)) r++;
98         // draw circle when timer 3 interrupt
99         LCD_Draw_Circle(x, y, r);
100        // show student-id
101        if (showStudentId == 1) {
102            LCD_ShowxNum(x-(fontSize/4), y-(fontSize/2), (u32)studentId[fontIndex], 1, fontSize, 1);
103        }
104    }
105    TIM_ClearITPendingBit(TIM3,TIM_IT_Update);
106 }

```

如上所述，timer 3 中斷裡是基礎題的主要邏輯，每次更新下一步前需清空 LCD 避免殘影（一開始還沒意識到，以為發生了點靈異現象），之後做下一刻結果的運算，最後將結果繪製在 LCD 上（感謝函示庫有畫空心圓的好東西）。

運算的想法為先看是否 enable（即下一刻應該向哪裡移動的意思），再看看是否照著該方向移動是否會撞到邊界，以做出相應的反應。向右主要就是中心橫軸加上當下半徑小於右邊界 $(x + r) < LCD\ width$ ，需要注意的是「=」不可以繼續向右，應該向左。因此一個像右的過程是直到碰到右邊界前都可以使 x 增加，遇到邊界就改變 enable flag 使方向發生改變，並在改變的當下向改變後的方向移動以避免出界。向左需考慮 flag 與中心橫軸減當下半徑是否大於左邊界 $(x - r) > 0$ 。左右方向講述完畢，上下方相也是相同原理，只要橫軸改縱軸即可。

對於縮小放大也需考慮 flag 與大小限制，縮小的功能我想要實現的是「小到一定程度就停止」的目的，而最小的圓圈內要可容納一個字，假設字大小（當作直徑）為 F，縮小後半徑最小要有 $\frac{F}{2}$ ，至於使用乘法而非除法的原因是浮點數運算在電腦上比較不精確，將運算控制在整數範圍以確保正確性（飄出半年競程仔資歷的惡臭），且 LCD 寬

度不可能達到變數 type 限制的 $2^8 - 1$ ，不會有溢位問題。放大須注意所有邊界的偵測，因為不可能預測使用者在哪放大，可能碰到的邊界都不同。

以上為下一步的運算，接下來只需將結果輸出到 LCD 即可，每 timer 3 發生中斷 $10\text{kHz} \times 500 = 50\text{ms}$ 做一次運算。運算本人不寫在 main 中原因為移動的時間難以掌控 (硬要寫也是可以在 while 中先 delay 一段時間後更新下一步)，但可讀性與可維護比較差，何不用較精確可控 timer 每固定時間後更新下一步，再說這些運算都是 $O(1)$ ，並不會有延遲的感受，這樣也可以讓 main 空下來做其他功能。

結果說明

根據基礎題的要求，以上 code 已經可以達成。可以在 LCD 任意處放大、縮小、上、下、左、右移動，且在任意行為觸碰到邊界都有各自的反應，並不會放任其超出邊界而發生意料之外的狀況。撰寫本作業秉持著「一次做一件事」的精神，如於移動過程中想縮小、放大就會停在按下 button 的位置開始動作，不會移動、放大縮小同時發生。

個人創意題

```
15 // remind user the key is already work (addition)
16 u8 ledFg = 0, showStudentId = 0, cnt = 0, ledCnt = 0;
17 // show student-id (addition)
18 u8 fontSize = 16, fontIndex = 0;
19 // fixed student-id (addition)
20 u8 studentId[8] = {0, 1, 2, 5, 7, 0, 2, 7};
```

以下說明會使用到的變數

ledFg	LED0 是否該亮的 enable flag
ledCnt	計數 timer 中斷次數使 LED0 點亮一段時間的計數變數
cnt	計數 timer 中斷次數得 user 按下 key0 多久的計數變數
showStudentId	是否該顯示 studentId 的 enable flag
studentId	本人學號的一維陣列
fontSize	顯示字體大小
fontIndex	studentId 陣列的 index

```

void EXTI0_IRQHandler(void) {
    // debounce
    delay_ms(10);
    if(WK_UP==1) {
        // remind user the function is working right now
        if (smallFg == 1) ledFg = 1;
        else {
            // turn circle small
            smallFg = 1, bigFg = 0;
            // disable any direction
            leftFg = 0, rightFg = 0, upFg = 0, downFg = 0;
        }
    }
    EXTI_ClearITPendingBit(EXTI_Line0);
}

```

上圖解釋驅使 ledFg 改變的方式，如果該 key 要做的事情已經正在做 (flag 已經是 enable)，代表不需要更動，使 LED0 亮起 (更改 flag 驅使 timer 計算中斷次數)。

```

32 // counter is 84M, 84M/8400=10kHz, 10kHz*100=10ms (addition)
33 TIM5_Int_Init(100-1,8400-1);

```

```

125 // timer 5 interrupt (addition)
126 void TIM5_IRQHandler(void) {
127     if(TIM_GetITStatus(TIM5,TIM_IT_Update)==SET) {
128         // shine 10ms*100=1000 ms
129         if ((ledFg == 1) && (ledCnt < 100)) {
130             LED0 = 0, ledCnt++;
131         } else {
132             LED0 = 1, ledCnt = 0, ledFg = 0;
133         };
134     }
135     TIM_ClearITPendingBit(TIM5,TIM_IT_Update);
136 }

```

上面兩張圖說明驅使 LED0 亮的始末，再更改 flag 後，10ms 內會被 timer 5 interrupt 捕捉，開始計算 100 次中斷 $10\text{kHz} \times 100 \times 100 \text{ 次} = 1\text{s}$ 達到亮一秒的目的，誤差時間 T 為 $0 \leq T \leq 10\text{ms}$ 。


```

30 // counter is 84M, 84M/8400=10kHz, 10kHz*5000=500ms (addition)
31 TIM4_Int_Init(5000-1,8400-1);

```

```

42 while(1) {
43     // if press key0
44     if (KEY0 == 0) {
45         // debounce
46         delay_ms(10);
47         while(KEY0 == 0);
48         // during 2 timer 4 interrupt => 500ms*2=1s
49         if (cnt >= 2) {
50             if (showStudentId == 0) {
51                 fontIndex = 0, showStudentId = 1;
52             }
53             else showStudentId = 0;
54         }
55     }
56     cnt = 0;
57 }

```

```

108 // show student-id
109 if (showStudentId == 1) {
110     LCD_ShowxNum(x-(fontSize/4), y-(fontSize/2), (u32)studentId[fontIndex], 1, fontSize, 1);
111 }

```

```

116 // timer 4 interrupt (addition)
117 void TIM4_IRQHandler(void) {
118     if(TIM_GetITStatus(TIM4,TIM_IT_Update)==SET) {
119         cnt++;
120         fontIndex = (fontIndex+1)%8;
121     }
122     TIM_ClearITPendingBit(TIM4,TIM_IT_Update);
123 }

```

以上四張圖所建立的功能是「長按 key0 大於 1s，開始以 500ms 一個數字的時間從頭播放學號」。第一張圖給出 timer 4 中斷時間 $10kHz \times 5000 = 500ms$ 。第二張圖片是 main 中的無限迴圈 (如之前所述，main 省下來可以做其他的事)，當 key0 確認 (以解彈跳方式確認) 被按下後被 while 卡住 (實現長按)，這段時間被 timer 4 中斷上數 cnt，假定放開 key0 時 cnt 為 2，key0 被按下總時長為 $500ms \times 2 = 1s$ ，達到長按 1s 的目的。長按後需視當下情況，如未顯示學號即顯示，否則關閉顯示，特別留意每次顯示時本人想要的是從頭開始，需要再開始顯示當下讓陣列 index 初始為零。

第三張圖是在 timer 3 interrupt 中的，與 LCD 顯示同步，提高較好的觀看體驗，避免螢幕先閃一下 (更新圓圈位置)，接著再閃一下 (更新學號)。數字的顯示設定為跟著圓圈中心移動，但字元的顯示為從左上開始，如果不校正，字元就會出現在圓圈中心的右下，因此需要抓一下座標 (橫軸除 4，縱軸除 2) 已經是本人認為最接近圓圈中心的調整了 (橫軸除 2 不會顯示在正中央可能源自於自元編碼的編排問題)。

本人原本想讓字元跟著圓圈放大縮小，但字元似乎有大小的絕對限制，如果大小不是 16，LCD 上不會有任何數字 (意思是大小變化為 $16 \Rightarrow 24 \Rightarrow 30$ ，非線性)。遙望電腦螢幕右下角，在深夜三點半的時刻，我選擇放過自己安然睡去。

心得

本人從這作業學到的最大收穫是 — 如何建立那該死的 include path，MDK5 甚麼都要自己拉著實有點反人類，身為一個手頭有函式庫模板想完全照搬過來零成本覆用的高階語言魔術師，誰會知道底層在不知名的陰暗小角落藏著那該死的 `stm32XX_tim.h` 之類的，我著實沒想到在這七小時的奮戰中我花了不下四小時研究如何搬運模板。

得力於模板代碼行數不多，本人 (樞門的算力資本家) 在 main 中逐行審視著每行 code 是否有存在的必要做刪減，但我又又沒想到 LCD 的運作需要依靠 uart 的初始化，這是一個令人難以理解的抽象事情，看上去腳位也沒有衝突，那只剩下 uart 初始化了 LCD 運作的必要動作，所以大人們建立的運作條件是「某人欲驅使 LCD，必先使 uart 致能？」，這思想著實高級，直到我意識到 Template 被我刪掉的 `uart_init` 可能是兇手時，一小時又這麼漂過去了，工程師的工作時長果然得從蒙騙自己人開始。

經過約半小時的奮戰，我生出了第一代的孩子 (代碼)，但圓圈的動作模式十分的抽象，上一刻可能在 LCD 左邊界，下一刻出現在中間偏右，又下一刻出現在左邊界，甚至走過還會留下殘影，這是被 Jojo 替身攻擊的樣子嗎？我 IQ 2_000_000 的大腦飛速的運轉著，在 0.000_000_001 秒意識到我將顯示 LCD 放在 500ms 的 interrupt，把移動放在 main 中，下一次顯示明顯移動了不少，且在顯示 LCD 前沒有 clear LCD，改動就是把所有事情放在 interrupt 中，在一進中斷就 LCD clear。

加分題的要求此時已經可以非常完美的成功運作，但是放大、縮小為按一下 `r+1`，我認知到當 demo 中圓圈大到撞到邊界這件事情會先讓我的手按到抽筋，於是乎再將第二代孩子送進模改場加工變成無限放大直至碰到任意邊界後停下。

此時此刻，深夜兩點半 (是個喝某家魚湯的好時間)，尚無睡意。對著第三代玩著玩著發現如果方為向下，再按下上下移動鍵，神奇的事情發生了，球撞到空氣牆頭也不回的離去了...好像哪裡怪怪的，不久，我發現了此行為是預設方向帶來的遺毒，必須多加個 if 保護，順便亮個 LED0 當作發現這問題的彩蛋 (又很給自己找事的設定 LED 亮一秒)。

看著第四代孩子內心的空洞 (空心圓) 我覺得可以再加點料，來點代表是自己的作品又漂亮的顯示—尊爵不凡的學號。這麼一個坑就這麼挖了，這又是一個有點難填的坑，我花了很久思考為何在模板中存在的 showNum 顯示不出字元，半夜的三點，無力的開發者試圖透過科技的力量 (gemini) 解決事情，它輕描淡寫的給出大小 16 後揮揮衣袖離去，徒留還不明白的傻子看著學號一個一個閃爍。終於，糊塗的旅人發現了深埋在模板中有著 if size==16 之類的事情，猜測字型大小是非線性的 (如果有人做出字形線性放大，我可能得親切的稱呼一聲爹)，所以第四版中 fontSize=12 才不會顯示任何東西，因為找不到相對應大小的字型，我沒有勤奮到為了每個字型大小聲成新的字型，再者可能先承受不住的是板子上的 ram 空間 (畢竟上年老師影片中可是因為編譯上限問題，連模板都 compile 不過)。

就這樣，第五版孩子誕生了，其代表著本人為 STM32 獻上的最高的工藝 (引用自 HONDA 對於 NSX 造車理念)，特色是字元顯示時圓保持最小狀態下仍然可以清晰看見字元的顯示而不會重疊，且圓始終涵蓋字元的顯示，字體也顯示於圓中心，不會發生連續按壓撞到空氣牆的行為 (當連續按壓有者好理解的 LED 閃爍提醒功能作動中)。

以上完成了此次試煉，耗時七小時開發，六小時報告，共十三小的時間，特別鳴謝 0x1(本人大名)鼎力相助，spotify 提供的情緒價值，gemini 一次的聰慧大腦租借，811 實驗室電腦的提供 (此次沒有在實驗室寫)，本家電腦，本田精神的傳承，沒有綠色乖乖但依舊乖乖地 STM32F407ZGT6。